План разработки AI Stock Market

Версия: 1.2

Дата: 2026-10-15

Статус: интерфейс A и сервер B реализованы по этапам и собраны в интеграционной ветке; выполняется приёмка бесплатного локального демо. Целевая итоговая ветка - main.

Репозиторий: https://github.com/nazek52/ai-stock-market

1. Цель и исходная точка

Создать веб-приложение для наблюдения за акциями: рыночный дашборд, регистрация, поиск и страницы компаний, личный список отслеживания, ежедневная AI-сводка и уведомления о заданных изменениях цены и объёма торгов. Интерфейс должен работать на компьютере и телефоне.

При подготовке версии 1.0 репозиторий был пуст. На 2026-10-15 есть интерфейс A, серверные модули B, контракты, тесты и локальные сервисы; исходники и согласованные интерфейсы сохраняются. Объединение в main и итоговые результаты проверок подтверждаются отдельным handoff, а не статусом плана.

План основан на видеоуроке JavaScript Mastery, его автосубтитрах и публичном коде автора. В уроке последовательно рассматриваются настройка (00:03), интерфейс и графики (00:16-01:09), регистрация и БД (01:09-02:54), ежедневная рассылка (02:54-03:38), поиск и акции (03:38-04:07), развёртывание (04:07-04:13). Список отслеживания предложен как отдельное задание в 04:13. Поэтому этапы списка и ценовых уведомлений ниже описаны как самостоятельная разработка, а не как уже готовый результат урока.

1.1 Границы первой версии

Входит в локальную v1: синтетический рынок и карточки акций, аккаунт, поиск, постоянный список, демонстрационная сводка по тестовым новостям, правила цены и проверяемые переходы порога, настройки, локальные MongoDB/Mailpit/Inngest. Письма остаются в Mailpit. Live-данные, Gemini и публичное размещение - дополнительный будущий режим.

Не входит: торговля, пополнение счёта, хранение денег, инвестиционные рекомендации, гарантированный поток биржевых сделок с минимальной задержкой, админ-панель и прогноз доходности. Эти функции требуют отдельных требований и решений по данным и правам доступа.

1.2 Принципы продукта

- Показатели и новости сопровождаются источником и временем обновления.
- AI пересказывает только полученные новости; не выдумывает цены, события или советы купить/продать.
- Ошибка внешнего API не ломает всю страницу: пользователь видит понятное состояние ошибки и время последнего успешного обновления.
- Пользователь управляет рассылкой и может отключить её.
- Частота уведомлений соответствует фактической доступности данных и ограничениям провайдера. Слово «мгновенно» не используется без подтверждённого источника событий и измеренной задержки.

1.3 Решение владельца от 2026-10-15: бесплатный демопроект

Проект показывает совместную работу участников A и B. Бюджет обязательных внешних сервисов - 0; новые облачные аккаунты, платные тарифы, домен и production не являются условиями приёмки v1. Используется собственный синтетический набор с явной меткой fixture. AI_MODE=fixture детерминированно собирает выбранные заголовки и не вызывает Gemini; такой результат нельзя представлять как ответ языковой модели.

Локальная среда: Node.js 24, MongoDB replica set, SMTP Mailpit, Inngest Dev Server. Команда npm run demo запускает единое приложение; сценарий - в docs/local-demo.md. Первая установка требует Интернета для пакетов и Docker-образов. TradingView остаётся необязательным внешним виджетом. Аутентификация, проверка прав, валидация, защита от дублей и настоящие записи в MongoDB сохраняются.

Для ручного показа ценового события предусмотрена страница /demo: явная кнопка проводит синтетический переход 99 -> 101 через отдельное временное правило текущего подтверждённого пользователя и локальную почту. После прогона правило удаляется; существующие правила не меняются. Это отдельный сценарий от preview сводки и автоматических тестов.

2. Архитектура и режимы

- Интерфейс и сервер: Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui.
- Аутентификация: Better Auth, сессии на сервере.
- Хранение: MongoDB; отдельные коллекции профилей, списков, правил уведомлений и истории отправок. Схемы и индексы описаны в этапе 2.
- Рыночные данные и новости: собственный fixture-адаптер для локальной v1; дополнительный Finnhub-адаптер включается отдельно. API-ключи остаются на сервере.
- Графики и рыночные виджеты: TradingView. Виджеты отображают данные TradingView; их обновление не следует путать с собственной системой событий приложения.
- Фоновые задания: локальный Inngest Dev Server; расписания демозапуска выключены по умолчанию. Облачный Inngest - будущая конфигурация.
- AI: детерминированный fixture-режим с источниками для локальной v1; Gemini-адаптер сохранён для будущих live-сводок.
- Email: Nodemailer через локальный SMTP Mailpit; письма не уходят внешним получателям. Внешний SMTP настраивается отдельно.
- Выпуск v1: локальный Next.js и Docker Compose для MongoDB/Mailpit/Inngest. Production preflight, Dockerfile и инструкция публикации сохранены как дополнительные инструменты.

2.1 Потоки данных

1. Браузер запрашивает страницу. Сервер проверяет сессию и отдаёт данные пользователя.

2. Поиск и карточка обращаются к выбранному серверному адаптеру: в демо возвращаются явно синтетические данные; в отдельно включённом live-режиме нормализуется и кэшируется ответ Finnhub.

3. Пользователь добавляет символ в список или создаёт правило. Сервер повторно проверяет сессию и записывает изменение в БД.

4. Фоновое задание читает список/правила, получает свежие данные, формирует сводку или проверяет порог, записывает результат и отправляет email.

5. Каждая отправка имеет идентификатор события для защиты от дублей и диагностики.

2.2 Минимальная модель данных

- `User/Profile`: идентификатор, email, имя, страна, инвестиционная цель, допустимый риск, предпочитаемая отрасль, часовой пояс, согласие на письма. Пароль и сессии принадлежат Better Auth.
- `WatchlistItem`: `userId`, нормализованный `symbol`, `companyName`, `createdAt`; уникальный индекс (`userId`, `symbol`).
- `AlertRule`: `userId`, `symbol`, тип (`price_above`, `price_below`, позднее `volume_above`), порог, статус, время создания, последнее срабатывание и период подавления повторов.
- `NotificationDelivery`: `userId`, тип, символ, ключ события, время, статус, код ошибки. Уникальный ключ события предотвращает повторную отправку.
- `DigestPreference`: включено/выключено, частота, часовой пояс, время отправки, последний успешно отправленный выпуск.

Схему уточнить перед миграциями; не хранить в БД ключи внешних сервисов и содержимое секретов.

3. Очерёдность и результаты

Этап 0. Уточнение продукта и подготовка (1-2 дня)

Цель: зафиксировать решения, от которых зависят API, уведомления и приёмка.

Задачи

- P0.1. Определить рынки и биржи первой версии, базовую валюту, язык интерфейса и целевую аудиторию.
- P0.2. Зафиксировать синтетический источник и интервалы демо: кэш 60 с, включённый обход 300 с, данные старше 900 с не вызывают событие. Тариф, лицензию и фактическую задержку Finnhub проверять перед будущим live-режимом.
- P0.3. Зафиксировать нулевой бюджет, локальные MongoDB/Mailpit/Inngest без облачных аккаунтов, порты и ограничения. Хостинг, регион и внешние аккаунты выбирать при отдельной публикации.
- P0.4. Утвердить основные экраны: вход, регистрация, дашборд, поиск, акция, список, правила, настройки.
- P0.5. Согласовать критерии приёмки первой версии и примерный набор тикеров для проверки.

Результат: короткий `docs/product-decisions.md` с принятыми решениями и открытыми вопросами; перечень сервисов и переменных окружения без значений секретов.

Готово, когда: локальные D1-D5 определены без внешнего аккаунта, синтетический источник и интервалы явно обозначены; будущие live-вопросы не блокируют демо.

Этап 1. Каркас приложения и качество сборки (1-2 дня)

Зависимость: решения P0.1 и P0.4.

Задачи

- P1.1. Создать Next.js App Router с TypeScript и закреплёнными совместимыми версиями зависимостей.
- P1.2. Подключить Tailwind CSS, базовые компоненты shadcn/ui, шрифты, тему и адаптивный каркас страниц.
- P1.3. Разделить серверный и клиентский код; добавить типы доменных моделей и единый модуль конфигурации.

- P1.4. Создать `.env.example`, `.gitignore`, команды `dev`, `build`, `lint`, `typecheck`; настроить автоматические проверки для новых изменений.
- P1.5. Добавить пустые состояния, общие страницы ошибок и базовую доступность навигации.

Результат: рабочий каркас с маршрутизацией и понятным запуском по README.

Готово, когда: проект запускается по инструкции; `build`, `lint` и проверка типов проходят; в Git нет секретов и локальных артефактов.

Этап 2. Учётные записи и профиль (2-3 дня)

Зависимость: этап 1, выбранная MongoDB.

Задачи

- P2.1. Подключить MongoDB и Better Auth, настроить регистрацию по email/паролю, вход, выход, серверную проверку сессии и хранение профиля/предпочтений отдельно от чувствительных данных аутентификации.
- P2.2. Создать формы страны, целей, риска и отрасли; передавать валидированные значения серверным операциям профиля.
- P2.3. Защитить личные маршруты и серверные операции; исключить доступ к чужим спискам и правилам через изменение идентификатора.
- P2.4. Обработать ошибки валидации, дубли email, истёкшие сессии и восстановление доступа в согласованном объёме.
- P2.5. Добавить настройки согласия на ежедневные письма и часового пояса.

Результат: пользователь может зарегистрироваться, войти, изменить допустимые настройки и выйти.

Готово, когда: перезагрузка страницы сохраняет сессию; чужие записи недоступны; формы показывают понятные ошибки; email-предпочтения сохраняются.

Этап 3. Дашборд и рыночные данные (2-3 дня)

Зависимость: этапы 1-2 и локальный рыночный адаптер. TradingView необязателен; Finnhub нужен только для live-режима.

Задачи

- P3.1. Собрать главную страницу: обзор индексов, карта рынка, основные котировки, новости, состояния загрузки и ошибки.
- P3.2. Интегрировать TradingView-виджеты с контролируемыми настройками символов, высоты, темы и адаптивности.
- P3.3. Добавить серверный клиент Finnhub: таймаут, обработка квот и ошибок, нормализация цены/времени/валюты, кэширование.
- P3.4. Показывать источник и время обновления там, где приложение отображает собственные данные.
- P3.5. Проверить мобильную вёрстку и работу при недоступности одного из поставщиков.

Результат: дашборд открывается после входа и показывает рыночную картину.

Готово, когда: виджеты читаемы на узком экране; отказ Finnhub или отдельного виджета не превращает весь экран в ошибку.

Этап 4. Поиск и страница акции (2-3 дня)

Зависимость: этап 3.

Задачи

- P4.1. Реализовать поиск по символу и названию с задержкой ввода, отменой устаревших запросов и ограничением частоты.
- P4.2. Нормализовать и валидировать символы, показать биржу и тип инструмента в результатах.
- P4.3. Создать маршрут `/stocks/[symbol]`: цена и изменение, графики, профиль компании, основные финансовые показатели и новости, если данные доступны.
- P4.4. Сделать понятные состояния «символ не найден», «данные недоступны» и «квота исчерпана».
- P4.5. Связать результаты поиска со страницей акции и будущим списком отслеживания.

Результат: пользователь находит акцию и открывает её страницу.

Готово, когда: поиск на синтетическом наборе работает по названию и тикеру; неизвестный символ показывает понятное состояние; Finnhub, если он включён отдельно, вызывается только сервером.

Этап 5. Личный список отслеживания (2-3 дня)

Зависимость: этапы 2 и 4. Это отдельная реализация финального задания из видео.

Задачи

- P5.1. Добавить модель `WatchlistItem` и уникальный индекс `(userId, symbol)`.
- P5.2. Создать серверные операции добавления и удаления с проверкой сессии, символа и владельца записи.
- P5.3. Подключить звёздочку в поиске и кнопку на странице акции; состояние должно приходить из БД, а не только из локального React-состояния.
- P5.4. Создать страницу списка: символ, компания, актуальная цена, изменение, дата добавления; сортировка и пустое состояние.
- P5.5. Обработать повторный клик и сбой API без потери согласованности интерфейса и БД.

Результат: персональный список, пригодный для рассылки и уведомлений.

Готово, когда: список переживает выход и повторный вход; пользователи видят только свои записи; дубль не создаётся.

Этап 6. Ежедневная AI-сводка (2-4 дня)

Зависимость: этапы 2, 4 и 5; локальный Inngest, детерминированная сводка и SMTP Mailpit. Ключи Gemini и облачного Inngest для демо не нужны.

Задачи

- P6.1. Подключить Inngest и создать задания приветственного письма и ежедневной сводки.
- P6.2. Для каждого подписанного пользователя получить новости по его списку; если список пуст, использовать общий рынок.
- P6.3. Передавать AI только нормализованные заголовки, ссылки, даты и краткие тексты; требовать фактического пересказа без выдуманных чисел и торговых советов.
- P6.4. Формировать письмо с датой, источниками и ссылкой на настройки; экранировать или санитизировать вывод AI перед вставкой в HTML.
- P6.5. Учитывать часовой пояс и согласие пользователя; обеспечить повторные попытки, защиту от повторной отправки и журнал результата.

- P6.6. Предусмотреть ручной тестовый запуск на одном тестовом аккаунте и возможность отключить расписание.

Результат: демонстрационная сводка по синтетическим новостям с источниками, preview и механизмом локальной доставки; автоматическое расписание включается отдельно.

Готово, когда: preview помечен тестовым; в контролируемом локальном прогоне тестовый пользователь получает один выпуск в Mailpit, отписанный не получает письмо, сбой одного адреса не останавливает остальных. Preview и доставка имеют отдельные проверки; live-AI не требуется.

Этап 7. Правила уведомлений (3-5 дней)

Зависимость: этапы 3, 5 и 6, синтетические наблюдения и локальные интервалы P0.2. Реальные биржевые события относятся к будущему live-режиму.

Задачи

- P7.1. Создать экран создания, изменения, приостановки и удаления правил уведомлений.
- P7.2. Добавить модель `AlertRule` и реализовать условия `price_above` и `price_below` с указанием валюты, источника цены и момента проверки.
- P7.3. Выбрать механизм проверки: провайдерские события, если доступны на выбранном плане, или периодические задания в пределах квот. Частоту документировать в интерфейсе.
- P7.4. Отправлять email при переходе через порог; записывать ключ события, последнее срабатывание и период подавления повторов.
- P7.5. Добавить `volume_above` только после проверки доступности и смысла данных об объёме для выбранных рынков.
- P7.6. Проверить нерабочие дни биржи, отсутствие котировки, повтор фонового задания и изменение правила между проверками.

Результат: управляемые пользователем уведомления с измеримой задержкой.

Готово, когда: контролируемый синтетический переход создаёт одно событие/письмо в локальной проверке; повтор не создаёт дубль, правило можно выключить. Статичная fixture-котировка не доказывает пересечение. Автотесты и ручная доставка в Mailpit имеют отдельные результаты.

Этап 8. Проверка, эксплуатация и выпуск (2-3 дня)

Зависимость: этапы 1-7.

Задачи

- P8.1. Проверить полный путь: регистрация -> поиск -> добавление -> сводка -> правило -> уведомление -> отписка.
- P8.2. Добавить автоматические проверки ключевой бизнес-логики: изоляция данных пользователей, уникальность списка, порог/повторы, выбор адресатов рассылки.
- P8.3. Проверить сборку, адаптивность, клавиатурную навигацию, ошибки API и ограничения квот.
- P8.4. Подготовить воспроизводимый локальный запуск Next.js/MongoDB/Mailpit/Inngest, проверить журналы и остановку заданий. Production-конфигурацию и требования домена сохранить как будущий дополнительный режим.
- P8.5. Задокументировать запуск, восстановление после сбоя, ротацию ключей и отключение рассылок.

Результат: совместный локальный демопроект A+B с понятным запуском и проверенными сценариями; целевая версия объединяется в main после финальных проверок.

Готово, когда: контрольный сценарий проходит локально, интерфейс A работает с сервером B, тестовые письма доступны в Mailpit, ключи не раскрыты, ограничения записаны, задания можно остановить. Production-сценарий потребуется только при отдельной публикации.

4. Распределение работы между двумя участниками

Участники обозначены как **A** и **B**. Эти обозначения закрепляют области ответственности независимо от имени и GitHub-аккаунта. A отвечает за пользовательские сценарии, интерфейс, доступность и приёмку поведения. B отвечает за сервер, БД, интеграции, фоновые процессы и выпуск. У каждой задачи P0.1-P8.5 один ведущий исполнитель; второй принимает интерфейс на границе своей зоны и помогает с интеграцией. Назначение ниже не отменяет критерии готовности соответствующего этапа.

4.1 Назначение задач

- **Этап 0:** A - P0.1, P0.4, P0.5 (рынки и сценарии, экраны, приёмка). B - P0.2, P0.3 (свежесть/права на данные, сервисы и квоты). Оба подтверждают решения D1-D5 до реализации зависимых задач.
- **Этап 1:** A - P1.1, P1.2, P1.5 (каркас Next.js, стили, навигация и состояния). B - P1.3, P1.4 (типы, конфигурация, `.env.example`, проверки CI). B передаёт A публичные типы до подключения форм и данных.
- **Этап 2:** A - P2.2, P2.4, P2.5 (анкеты, сообщения об ошибках, настройки писем). B - P2.1, P2.3 (Better Auth, MongoDB, сессии, профиль, защита). Сначала B согласует контракт профиля и операций с A; затем A подключает формы.
- **Этап 3:** A - P3.1, P3.2, P3.4, P3.5 (дашборд, виджеты, время обновления, адаптивность). B - P3.3 (серверный адаптер Finnhub и нормализованные типы). До готовности адаптера A использует фикстуры той же формы ответа.
- **Этап 4:** A - P4.1, P4.3, P4.4, P4.5 (поиск и страница акции). B - P4.2 (валидация символа, биржа и тип инструмента в нормализованном ответе). A подключает серверный поиск после handoff B.
- **Этап 5:** A - P5.3, P5.4, P5.5 (кнопки, страница списка, состояние интерфейса при сбое). B - P5.1, P5.2 (модель, индекс, защищённые операции). Контракт операций add/remove/read согласовать до параллельной работы.
- **Этап 6:** A - P6.4, P6.6 (шаблон письма, отображение источников и настроек, тестовый ручной прогон). B - P6.1, P6.2, P6.3, P6.5 (Inngest, новости, AI, расписание, повторы и журнал). A передаёт B утверждённый шаблон и требования к данным письма.
- **Этап 7:** A - P7.1, P7.6 (экран правил и приёмочные сценарии). B - P7.2, P7.3, P7.4, P7.5 (модель, проверка порогов, отправка и защита от дублей). B передаёт A контракт правила и фактическую частоту проверки.
- **Этап 8:** A - P8.1, P8.3, P8.5 (сквозной сценарий, интерфейс/доступность, пользовательская и эксплуатационная документация). B - P8.2, P8.4 (автоматические проверки, локальный выпуск, журналирование; production как дополнительный режим). Выпуск принимают оба: A подтверждает пользовательский путь, B - работоспособность фоновых задач и инфраструктуры.

4.2 Порядок совместной работы

1. B в начале этапа B публикует типы, схему ответа и ошибки для серверных операций; A подтверждает, что они покрывают экран. Для чисто интерфейсных задач A может начать с фикстур.

2. Каждый работает в своей ветке и открывает отдельный PR. Общие файлы (`package.json`, маршруты, типы, схемы, глобальные стили) меняются после короткого согласования; один файл одновременно редактирует один участник.

3. Handoff оформляется по `agents/contracts/handoff.md`. Следующий участник запускает зависимые проверки, прежде чем считать интеграцию готовой.

4. Совместная проверка нужна на границах P2/P3, P4/P5, P5/P6, P6/P7 и перед выпуском. Не начинать рассылку реальным пользователям до проверки согласия, отписки и ограничения повторов.

4.3 Исходный ориентир по календарю

Оценки ниже сохранены из версии 1.1 для понимания порядка работ. На 2026-10-15 это не оценка оставшегося срока: реализация уже существует, текущая задача - совместная интеграция и проверка локального демо.

- Дни 1-2: совместное уточнение этапа 0; A начинает каркас, B готовит контракты данных и окружение.
- Дни 3-6: A собирает интерфейс входа и дашборд на фикстурах; B завершает авторизацию и рыночный адаптер.
- Дни 7-10: A делает поиск, страницу акции и список; B завершает серверные операции списка и основу рассылки.
- Дни 11-15: A готовит шаблон письма, экран правил и сквозные сценарии; B завершает AI-сводку и движок уведомлений.
- Дни 16-19: оба исправляют интеграционные дефекты; B готовит локальный демозапуск, A принимает интерфейс и документацию.

Это порядок, а не обещанная дата выпуска. Оценка составляет примерно **12-16 рабочих дней для A** и **14-19 для B**; календарный срок **16-20 рабочих дней** при готовых доступах и своевременных решениях. Узкое место - интеграции и фоновые задачи B. Если B задерживается, A готовит фикстуры, доступность, тестовые сценарии и документацию, не подменяя контракт данных.

5. Вехи, сроки и приоритет

Исходная оценка версии 1.1 для одного разработчика составляла 16-25 рабочих дней при готовых доступах. Она сохранена как история планирования; оставшийся объём локальной приёмки определяется текущими проверками, а не этим календарём.

- **M1 - просматриваемый прототип:** этапы 0-3. Есть каркас, аккаунт и дашборд.
- **M2 - полезная первая версия:** этапы 4-6. Есть поиск, страницы акций, постоянный список и ежедневная сводка.
- **M3 - локальная v1 A+B:** этапы 7-8. Есть правила, контролируемые проверки событий и локальный выпуск. Облачные аккаунты и live-данные для этой вехи не обязательны.

Если сроки ограничены, выпускать M2 отдельно. Локальные уведомления нельзя обозначать проверенными без контролируемого синтетического перехода и отдельной проверки доставки. Объём P7.5 остаётся отложенным. Реальную задержку live-уведомлений проверяют только при подключении реального источника.

6. Сквозные критерии готовности

Каждая задача завершена, только если:

1. Изменение привязано к пункту плана и имеет проверяемый результат.
2. Секреты остаются вне репозитория; публичные переменные не содержат приватных API-ключей.
3. Доступ к личным данным проверяется на сервере для каждого чтения и изменения.
4. Ошибки поставщиков данных, AI и почты обрабатываются и записываются без раскрытия секретов.
5. Добавлены необходимые проверки поведения, а сборка, линтер и типы проходят.

6. Обновлены README, `.env.example` и документы, если изменились запуск, данные или контракты.

7. Агент или разработчик передал результат по шаблону `agents/contracts/handoff.md`.

7. Решения локальной v1 и будущего live-режима

ID	Вопрос	Принято для локальной v1
D1	Что показывает v1?	Собственные синтетические US equity/USD, русский интерфейс; live-покрытие отдельно.
D2	Как проверять данные демо?	Кэш 60 с, включённый обход 300 с, stale 900 с; реальные права и задержка только для live.
D3	Когда отправлять сводку?	Настраиваемое локальное время пользователя; один выпуск в календарный день его часового пояса.
D4	Куда отправляются письма v1?	Только в локальный Mailpit; внешняя почта не требуется.
D5	Где запускается v1?	Локально: Next.js и Docker Compose MongoDB/Mailpit/Inngest, бюджет внешних сервисов 0.

До принятия решения агент фиксирует предположение в `docs/product-decisions.md` и не обещает пользователю неподтверждённые свойства данных или сервиса.

8. Ссылки

- Видео и временные разделы: https://youtu.be/gu4pafNCXng
- Публичный код автора: https://github.com/adrianhajdin/signalist_stock-tracker-app
- Рабочий репозиторий: https://github.com/nazek52/ai-stock-market

9. Журнал изменений

- 2026-10-15, v1.2: владелец выбрал бесплатную локальную v1 для демонстрации A+B; обновлены репозиторий, фактическая исходная точка, режимы данных/AI/почты, зависимости и критерии приёмки. Все 47 задач и распределение ролей сохранены. Публичный production перенесён в дополнительный будущий режим.
- 2026-10-14, v1.1: задачи P0.1-P8.5 распределены между двумя участниками, добавлены handoff, порядок параллельной работы и календарный ориентир.
- 2026-10-14, v1.0: создан единый план по видео; отдельно описаны список отслеживания и уведомления, которых нет в завершённом коде урока.